Literals and Type Conversions

1. Literals.

- Java Literals are syntactic representations of boolean, character, numeric, or string data.

Example: 4 is an integer literal.

Identifiers: Identifiers are the names of variables, methods, classes, packages and interfaces.

- Octal values are denoted in Java by a leading zero. Normal decimal numbers cannot have a leading zero.
- Thus, the seemingly valid value 09 will produce an error from the compiler, since 9 is outside of octal's 0 to 7 range.
- You signify a hexadecimal constant with a leading zero-x, (0x or 0X).
- The range of a hexadecimal digit is 0 to 15, so A through F (or a through f) are substituted for 10 through 15.
- When a literal value is assigned to a byte or short variable, no error is generated if the literal value is within the range of the target type. An integer literal can always be assigned to a long variable.
- However, to specify a long literal, you will need to explicitly tell the compiler that the literal value is of type long.
- You do this by appending an upper- or lowercase L to the literal.
- Beginning with JDK 7, you can also specify integer literals using binary. To do so, prefix the value with 0b or 0B.
- For example, this specifies the decimal value 10 using a binary literal:

```
int x = 0b1010;
```

```
int x = 123_456_789;
```

- the value given to x will be 123,456,789. The underscores will be ignored.
- Floating-point literals in Java default to double precision.
- To specify a float literal, you must append an F or f to the constant.
- Hexadecimal floating-point literals are also supported, but they are rarely used.
- They must be in a form similar to scientific notation, but a P or p, rather than an E or e, is used.
- For example, 0x12.2P2 is a valid floating-point literal. The value following the P, called the binary exponent, indicates the power-of-two by which the number is multiplied.
- Therefore, 0x12.2P2 represents 72.5.

```
double num = 9_423_497_862.0;
```

- the value given to num will be 9,423,497,862.0

double num = 9_423_497.1_0_9;

- Boolean literals are simple. There are only two logical values that a boolean value can have, true and false.
- The values of true and false do not convert into any numerical representation.
- The true literal in Java does not equal 1, nor does the false literal equal 0.
- As you may know, in some other languages, including C/C++, strings are implemented as arrays of characters.
- However, this is not the case in Java. Strings are actually object types.
- Because Java implements strings as objects, Java includes extensive string-handling capabilities that are both powerful and easy to use.
- The remainder a % b is defined such that (a / b) * b + a % b is always equal to a :

```
System.out.println(-15 % 2);    // -1

System.out.println(15 % -2);    // 1
```

2. Type Conversion and Casting.

Java's Automatic Conversions.

- When one type of data is assigned to another type of variable, an automatic type conversion will take place if the following two conditions are met:
    - The two types are compatible.
    - The destination type is larger than the source type.
- Java also performs an automatic type conversion when storing a literal integer constant into variables of type byte, short, long, or char.

Casting Incompatible Types.

- Although the automatic type conversions are helpful, they will not fulfill all needs.
- For example, what if you want to assign an int value to a byte variable? This conversion will not be performed automatically, because a byte is smaller than an int.
- This kind of conversion is sometimes called a narrowing conversion, since you are explicitly making the value narrower so that it will fit into the target type.
- For example, the following fragment casts an int to a byte. If the integer's value is larger than the range of a byte, it will be reduced modulo (the remainder of an integer division by the) byte's range.
    - int a;
    - byte b;
    - b = (byte) a;
- A different type of conversion will occur when a floating-point value is assigned to an integer type: truncation.

Automatic Type Promotion in Expressions:

- int a = 257;
- byte b = (byte) a;

- When the value 257 is cast into a byte variable, the result is the remainder of the division of 257 by 256 (the range of a byte), which is 1 in this case.
    - byte a = 40;
    - byte b = 50;
    - byte c = 100;
    - int d = a * b / c;
- The result of the intermediate term a * b easily exceeds the range of either of its byte operands.
- To handle this kind of problem, Java automatically promotes each byte, short, or char operand to int when evaluating an expression. This means that the subexpression a*b is performed using integers, not bytes.

- byte b = 50;
- b = b * 2; // Error! Cannot assign an int to a byte!

- The code is attempting to store 50 * 2, a perfectly valid byte value, back into a byte variable.
- However, because the operands were automatically promoted to int when the expression was evaluated, the result has also been promoted to int.

The Type Promotion Rules:

- Java defines several type promotion rules that apply to expressions.
- They are as follows: First, all byte, short, and char values are promoted to int, as just described.
- Then, if one operand is a long, the whole expression is promoted to long. If one operand is a float, the entire expression is promoted to float. If any of the operands are double, the result is double.
    - class Promote {
    - public static void main(String args[]) {
    - byte b = 42;
    - char c = 'a';
    - short s = 1024;
    - int i = 50000;
    - float f = 5.67f;
    - double d = .1234;
    - double result = (f * b) + (i / c) - (d * s);
    - System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));
    - System.out.println("result = " + result);
    - }
    - }

- Let's look closely at the type promotions that occur in this line from the program:
    - double result = (f * b) + (i / c) - (d * s);
- In the first subexpression, f * b, b is promoted to a float and the result of the subexpression is float.
- Next, in the subexpression i/c, c is promoted to int, and the result is of type int. Then, in d * s, the value of s is promoted to double and the type of the subexpression is double.
- Finally, these three intermediate values, float, int and double, are considered.
- The outcome of float plus an int is a float. Then the resultant float minus the last double is promoted to double, which is the type for the final result of the expression.